The Problem

Problem Statement: The Friction of Shared Expenses

Executive Summary

Managing group expenses is a socially awkward and mathematically tedious process. Despite the frequency of group activities—such as traveling, sharing an apartment, or dining out—the methods for tracking, calculating, and settling debts remain fragmented, leading to financial loss and strained personal relationships.

Core Challenges

1. Financial Leakage and Inaccuracy

Individuals who cover the initial bill often end up losing money due to "invisible costs." These include:

- **Uncalculated Add-ons:** Tips, taxes, and service fees are frequently omitted during manual splits.
- **Rounding Errors:** Inconsistent methods for handling decimals lead to small but cumulative losses.
- **Missing Records:** The lack of immediate receipt digitization results in forgotten expenses and lost proof of payment.

2. Logistical and Mathematical Complexity

Splitting costs is rarely a simple division by the number of participants. Complexity arises from:

- **Mixed Payment Methods:** Groups often use a combination of cash, digital wallets, and credit cards, making it hard to track the flow of money.
- **Unequal Consumption:** Traditional "split-even" methods are unfair when participants have vastly different consumption patterns (e.g., someone not ordering alcohol or appetizers).
- **External Constraints:** Many establishments refuse to split checks, forcing one person to take the financial risk for the entire group.

3. Social Friction and Communication Breakdown

The most significant burden is psychological. The "debtor-creditor" dynamic creates several social pain points:

- **The "Debt Collector" Paradox:** People often feel embarrassed or "cheap" when asking friends for their money back, leading them to abandon the debt altogether.
- **Information Asymmetry:** Discrepancies in memory regarding who paid for what lead to misunderstandings and mistrust.

- **Forgetfulness and Procrastination:** Without automated reminders, debtors often genuinely forget their obligations, forcing the creditor to send awkward manual follow-ups.

The Opportunity

There is a clear need for a **neutral, transparent, and automated intermediary** that removes the emotional weight from financial transactions, ensures mathematical precision, and provides a single "source of truth" for group finances.

Requirements Engineering

Table of Content

Requirement Engineering..... 1
Target User(s).....1
Functional Requirements..... 2
Authentication Module.....2
Group Module.....2
Expenses Module (Core).....3
Non-Functional Requirements.....4
Implementation Details.....5

Requirement Engineering

This section aims to provide a clear understanding of the project's scope and specifications. It describes the intended users and lists the functional and non-functional requirements, demonstrating how the final product effectively addresses these needs.

Target User(s)

As described in The Problem section, this application aims to help any person that needs a ***source of truth*** when sharing expenses with other persons, in small or large groups. Thus, we aim for users that are involved in activities such that expenses are expected to be shared somehow equally or fair between the group. This include people that:

- Likes to travel
- Are living in shared houses/apartments/floors.
- Are involved in social groups that require expense tracking.
- And have basic knowledge of using daily applications in Smartphones / PCs.

Every user has the same possibility as any other uses(without taking in count premium accounts), but, the application identifies two distinct user roles as follows:

1. Administrator: this person is the one who manages the group, being able to performn actions like changing attributes of the group, inviting/deleting people from the group, and any other action that would be expected only to be performed by the leader of the group. Lastly, perform **group member** actions.
2. Group member: aside from the administrator, this user is the one that participates normally in the group, creating expenses, being able to see other expenses on the group, and participating in the settlement of the debt owed.

Functional Requirements

The functional requirements define all the logic business actions that the application/system is able to perform, with or without user interaction. The functional requirements are going to be divided into **functional modules**, in order to separate them depending on the domain they are working on.

Authentication Module

This module handles the ability of an user to access into the system, and to make sure that non-authenticated users cannot perform other actions rather than log in into the application.

Code	Name	Description	Constraints
FR - 001	Login	The user should be able to login into the app, by giving his credentials (email, password), and getting access to Groups/Expenses sections.	—
FR - 002	Logout	The user should be able to logout of the app, and be redirected to the main / login page.	Authentication required.

Group Module

This module handles the ability of managing the groups, with crucial actions such as creating, seeing groups, and also, managing the users inside the group. Depending on the actions performed, administrator access could be also required from the user.

Code	Name	Description	Constraints
FR - 003	Create Group	The user should be able to create a new group, inserting all necessary information (name, description), and getting administrator access to it.	Authentication required.
FR - 004	List Groups	The user should be able to see a list of the groups he owns and he is part of.	Authentication required.
FR - 005	Search Group	The user should be able to see the information of a specific single group.	Authentication required Group access required.
FR - 006	List Member Groups	The user should be able to see a list of the users that are included in the group.	Authentication required Group access required.
FR - 007	Get Invite Code	The user should be able to get an invitation code, so other users get access to the group from the link.	Authentication required Group access required. Administrator access.

FR - 008	Add user to the group	The user should be able to insert another user into one of his groups, by specifying the email of the user.	Authentication required Group access required. Administrator access.
FR - 009	Delete user from the group	The user should be able to delete an existing user from the group's list of users.	Authentication required Group access required. Administrator access.

Expenses Module (Core)

This module handles the core abilities of the application, letting users create expenses within the groups they are part of. And thus, getting to know what the debts are, and how they are expected to settle those debts.

Code	Name	Description	Constraints
FR - 010	Create Expense	The user should be able to create a new expense in a group he is part of, inserting all information needed (cost, description, and users for splitting the debt)	Authentication required Group access required.
FR - 012	List Expenses	The user should be able to see a list of all the expenses that have been created within the group.	Authentication required. Group access required.
FR - 013	Read Expense	The user should be able to check the full details of a single expense.	Authentication required. Group access required.
FR - 014	List debts	The user should be able to see a list of each debt between any pair of persons included in the group.	Authentication required. Group access required.
FR - 015	Settle a debt	The user should be able to settle to zero the debt he owes to the group.	Authentication required. Group access required.

Non-Functional Requirements

The non-functional requirements specify the constraints and qualities that the application should have, in order to be successfully used by all users, and also to be implemented and maintained correctly in the whole SDLC (*Software Development Life Cycle*). This application has the following NFR (Non-Functional Requirements):

Code	Category	Description
NFR - 001	Usability	The application should be accessible from any modern device, including smartphones, laptops, tablets, and others.
NFR - 002	Usability	The error messages and validations should be easy to understand and easy to read for all users.
NFR - 003	Security	The application should implement authentication methods using private user credentials.
NFR - 004	Security	Each user session should expire automatically after a determined amount of time (that should be able to be configured inside or outside the application).
NFR - 005	Security	Each action to be performed should be checked for user roles to be executed.
NFR - 006	Efficiency	The application should handle each request in a period of time no more than 3-5 seconds.
NFR - 007	Efficiency	The application should be able to handle more than 100 users at a time.
NFR - 008	Usability	The user interfaces should have a coherent design throughout the whole application.
NFR - 009	Usability	All main forms should include proper validation messages.
NFR - 010	Usability	All basic actions as login into the system, creating expenses, seeing list of groups/expenses, should be able to be performed by users without advanced capacitation.

Implementation Details

The application aims to satisfy each FR and NFR by implementing an architecture that makes it possible to handle each request in correct order, following good code practices, validations, security measures, and other qualities. For this, the application is structured in two main parts: **Backend** and **Frontend.**

The backend side aims to handle the business logic, data handling, security, authentication measures. For these, it uses an approach called DDD (Domain-Driven Design), in which each main use case is modeled in its specific layer, being able to implement those in any web-based framework. Then, it includes a web-framework as FastAPI that lets the application handle user requests in an asynchronous way, making it suitable for many requests at a time by the users, and making it possible for validations of types to happen in real time. For last, it uses an ORM with a SQL database, making it possible to save all data in proper formats, suitable for fast queries and future scalability.

On the other side, the frontend side aims to handle all user interfaces, and since it's separated from the backend side, it only focuses on bringing the greatest usability to the user, comfort in visiting the application, and dealing with getting user input, and showing information as organized as possible. The frontend also makes sure that standard procedures such as: error validation, proper messages to the user and user navigation, are correctly implemented throughout the whole app. For this, it uses a web-based framework called NextJS, this framework brings the opportunity to make large-scale applications on the future, while ensuring proper code practices for the prototype version. Also, gives easy implementations on routing, connecting to the Backend application, reusing components and dealing with styles (colors, themes, fonts, etc).

Non-Technical Details

Table of Content

Non-Technical Details **1**
Technical Decisions 1
Stack chosen 1
Architecture overview 2
Explanation of key design decisions 3
Trade-off considered 3
Assumption and Constraints 4
Assumptions made during development 4
Simplifications introduced 4
Known limitations of current implementation 4
Requirements intentionally postponed 5
Future improvements 5
Features to implement next 5
Technical debt 6
Security improvements 6

Non-Technical Details

This section aims to bring some understanding about non-technical details of the application. Covering basic topics like technologies used, architecture of the application, limitations, constraints, making it possible for any developer to understand the application model and to give proper information for the whole *Software Development Life Cycle* for this project.¡

Technical Decisions

Stack chosen

The stack of the application is divided in two main parts, **Backend** and **Frontend** application. Each of these parts have their own stack, as follows:

Backend:

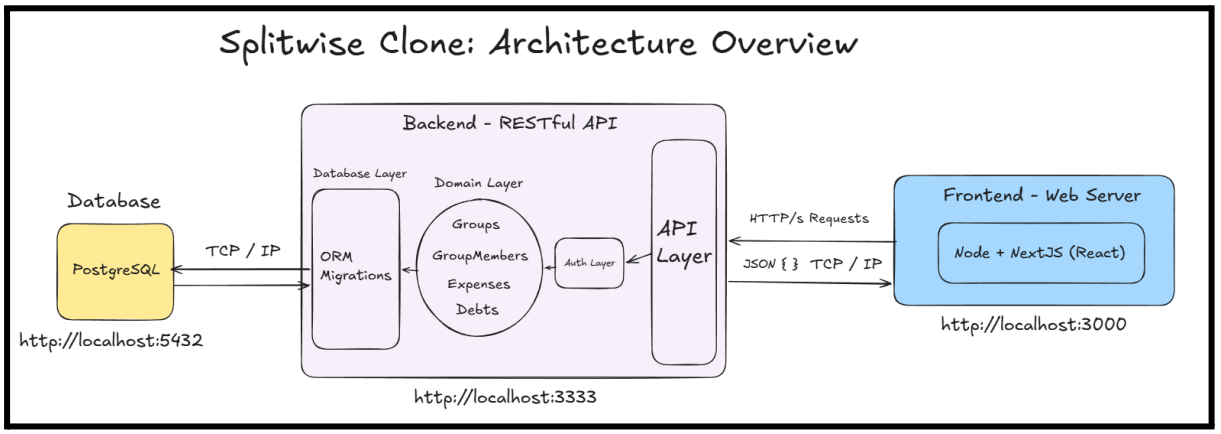
- **Web-based framework:** FastAPI
- **SQL Database:** PostgreSQL
- **ORM:** SQLAlchemy & Alembic

Frontend:

- **Web-based framework:** NextJS (React)

Architecture overview

The architecture for this application, if we talk about functional components, is divided into three main parts: Backend, Frontend and Database. This can be schematized as follows:



The architectural flow initiates when the user accesses the application via a web browser, where the **Frontend server** renders the user interface. User interactions—such as logging in or managing expenses—trigger HTTP requests to the **Backend server**.

Upon reaching the backend, requests are first handled by the **API Layer**, which routes them to the appropriate endpoint. Here, the **Auth Layer** intercepts protected routes to verify user credentials before processing continues. Validated requests are then passed to the **Domain Layer**, which executes the core business logic (e.g., for Groups or Expenses).

To persist or retrieve data, the Domain Layer utilizes an **ORM (Object-Relational Mapper)**. The ORM translates domain actions into SQL queries and communicates via TCP/IP with the **Database instance** (e.g., PostgreSQL). Finally, the query results traverse back through the layers, generating an HTTP Response that the Frontend uses to update the UI or display messages to the user.

Explanation of key design decisions

During the design phase, the team made several critical architectural decisions to ensure the prototype adheres to best practices and remains scalable. These decisions include:

a. Adoption of Domain-Driven Design (DDD). Instead of starting with database schemas or API implementation, the team adopted a **Domain-Driven Design** approach. This strategy prioritized the definition of core business entities, data attributes, and main use cases independently of the infrastructure. This allowed the team to model the domain logic first, postponing technical implementation details (such as persistence mechanisms or framework choices) until the business requirements were clearly defined.

b. Decoupled Frontend Architecture. The team evaluated whether to use server-side rendering within the backend or to implement a completely separate frontend application. The decision was made to **separate the Frontend from the Backend**. Despite the additional complexity, this approach ensures a strict separation of concerns and improves scalability. It allows the backend to focus purely on API logic and data handling, while enabling the use of a modern JavaScript framework to create a richer, more responsive user interface, aligned with modern web standards.

c. Technology Stack Selection. The technology stack was selected based on a balance between **development velocity and team expertise**.

- **Backend: FastAPI** paired with **SQLAlchemy** was chosen for its high performance, type safety, and seamless integration with domain models.
- **Frontend: Next.js** was selected as the client-side framework. This choice leverages the team's existing proficiency, ensuring rapid prototyping without sacrificing code quality or reliability.

d. Persistence Strategy (Relational Database). Although this is a prototype, persistent data storage is essential for future feature development. The team opted for a **Relational Database** (RDBMS) rather than a NoSQL solution. This de

Trade-off considered

Initial Complexity vs. Long-term Maintainability (DDD Approach): Adopting Domain-Driven Design required a significant upfront investment in defining domain models and abstraction layers, slowing down the initial development phase. However, this trade-off was accepted to ensure the business logic remains decoupled from infrastructure, facilitating future changes and testing.

Operational Overhead vs. UX Flexibility (Decoupled Architecture): Separating the Frontend (Next.js) from the Backend (FastAPI) introduces complexity in deployment, state management, and API communication (e.g., CORS). We accepted this overhead to gain the ability to build a rich, interactive user interface and to allow independent scaling of the client and server applications.

Abstraction Overhead vs. Data Integrity (ORM Usage): Using SQLAlchemy adds a layer of abstraction that can introduce slight performance overhead compared to raw SQL execution. This trade-off was deemed necessary to prioritize development speed, code readability, and database portability during the prototyping phase.

Assumption and Constraints

Assumptions made during development

Throughout the whole development process, the following assumptions were made.

- It is assumed that the user has a stable and continuous internet connection, as there is no offline mode available.
- It is assumed that users will access the application using up-to-date modern web browsers (e.g., Chrome, Firefox, Edge).
- It is assumed that the system load—including data volume, concurrent sessions, and requests per second—will not exceed a standard threshold of approximately 50 concurrent users.
- It is assumed that the deployment environments support Python and JavaScript runtimes and provide sufficient memory to host the Backend and Frontend services respectively.

Simplifications introduced

- The application is deployed on free-tier cloud platforms, which offer limited RAM and CPU resources compared to production-level instances.
- User profiles are kept simple and do not support custom avatar uploads.
- Users cannot manually search for or join public groups; joining a group strictly requires an invitation link.
- Integration with external payment providers to settle debts was excluded.
- Authentication relies solely on a basic email and password method, utilizing JWT (JSON Web Tokens) for session management.
- A real-time chat feature was left out to avoid the additional architectural overhead of implementing WebSockets.

Known limitations of current implementation

- Groups are restricted to a single currency. It is not possible to add expenses with different currencies within the same group.
- The system currently lacks automated notification mechanisms, such as email alerts or push notifications.
- The product is strictly a web application and does not have native mobile versions for Android or iOS.

Requirements intentionally postponed

During the development of the MVP prototype, the team intentionally postponed certain functional requirements deemed non-essential for this initial release. Omitting these features limits the application's overall scope, but it ensures the product still successfully meets its core criteria and primary objectives.

The following requirements were intentionally deferred:

- Editing user profiles and deleting accounts.
- Leaving a group or assigning administrator privileges to other members.
- Editing and deleting existing expenses.
- Attaching notes or photos (e.g., receipts) to expenses.
- Receiving automated notifications for new expenses, payment settlements, and invitations.
- Integrating external payment platforms to settle debts.
- Managing a user contacts (friends) list and implementing an internal chat system.
- Supporting multiple currencies within the application.
- Implementing third-party authentication methods (e.g., OAuth).

All of these requirements are planned for future, production-ready versions of the application, pending proper analysis and the allocation of sufficient time and resources.

Future improvements

Features to implement next

The immediate priority for future development is to implement the features that were intentionally postponed (see the *Requirements Intentionally Postponed* section). Once those core requirements are met, the team plans to introduce advanced features to enhance the user experience and increase the product's commercial viability.

The proposed roadmap includes:

- **Graphical Timelines:** Visualizing the expense history of a group over time.
- **Expense Categories:** Adding specific tags (e.g., groceries, tickets, food) to improve expense tracking and filtering.
- **Receipt Scanning:** Implementing a feature to scan bills, automatically extracting products and prices to assign specific debts.
- **Recurring Payments:** Setting up easy-to-use templates for recurring group expenses (e.g., monthly rent or subscriptions).
- **Global Balance Dashboard:** Allowing users to view their total combined balance across all active groups.
- **Currency Conversion:** Automatically converting debt amounts based on real-time exchange rates.
- **Advanced Notifications:** Sending alerts via email or SMS, paired with a preference center for users to manage their notification settings.

- **Smart Reminders:** Setting reminders for unsettled debts and syncing them with external cloud calendars (e.g., Google Calendar).
- **Monetization Strategy:** Introducing premium subscription plans alongside the base free tier.
- **Native Mobile Application:** Developing dedicated apps for iOS and Android environments.

Many of these advanced features go beyond the standard offerings of competitors like Splitwise. By implementing them, the objective is to significantly increase user satisfaction, differentiate the application, and successfully position it in a competitive market environment.

Technical debt

As the prototype was developed with a focus on delivering core functionalities within limited timeframes, certain technical shortcuts were taken. To ensure the application scales sustainably, the following technical debt must be addressed:

- **Frontend Testing Coverage:** While the Backend currently implements automated testing using Pytest, the Frontend lacks comprehensive unit and end-to-end tests. Establishing a robust testing suite using tools like Jest and Cypress for the client-side application is a priority to prevent UI regressions when adding new features.

- **CI/CD Pipeline Enhancements:** Although the core deployment processes are successfully automated via CI/CD pipelines, these workflows currently lack advanced validation stages. Integrating automated security scanning (SAST), strict code quality gates, and automated frontend testing into the existing pipeline is necessary to ensure greater robustness before production releases.

- **Database Indexing and Query Optimization:** As the data volume grows (especially with future features like 'graphical timelines' and 'expense history'), current SQLAlchemy ORM queries might become a bottleneck. A thorough review of database indexes and query optimization is needed.

- **Centralized Logging and Monitoring:** The system currently relies on basic console logs. Integrating a centralized error tracking and monitoring service (such as Sentry or Datadog) is necessary to proactively identify and resolve production issues.

Security improvements

Transitioning from a prototype to a market-ready product requires hardening the application's security posture and ensuring compliance with international data protection laws, particularly the **GDPR (General Data Protection Regulation)** for European standards. Future security enhancements include:

GDPR Compliance Mechanisms: > **Right to be Forgotten:** Implementing strict data deletion protocols to ensure that when a user requests account deletion (a postponed

requirement), all associated Personally Identifiable Information (PII) is permanently removed or fully anonymized.

- **Data Portability and Consent:** Adding features for users to export their data and implementing explicit, granular consent mechanisms for cookies and data processing.

API Rate Limiting and Throttling: Implementing strict rate limits on the FastAPI endpoints (especially authentication routes) to mitigate brute-force attacks and Denial of Service (DoS) attempts.

Data Encryption at Rest: While data in transit is protected via HTTPS, sensitive user data and financial records within the PostgreSQL database need to be encrypted at rest to protect against potential database breaches.

Advanced Security Headers and CORS: Hardening the Next.js frontend by implementing strict Content Security Policies (CSP) and configuring precise Cross-Origin Resource Sharing (CORS) rules in the backend to prevent XSS (Cross-Site Scripting) and CSRF (Cross-Site Request Forgery) attacks.

Multi-Factor Authentication (MFA): Given that the application handles financial tracking, adding an optional 2FA/MFA layer will provide critical protection for user accounts beyond the current basic JWT implementation.

Testing Plan

Table of Content

Testing Plan **1**
Overview 1
Test Types 1
Test FIles Overview 2
Tools 2
Manual testing 3

Testing Plan

This section describes the testing strategy used to validate the Splitwise Clone application. The goal is to ensure correctness of both the core business logic and the HTTP API across normal operation and edge cases.

Overview

The test suite is organized in two levels. The first level consists of domain unit tests, which test the business logic in complete isolation from the database and the web framework, using in-memory repositories and plain Python objects. The second level consists of API integration tests, which test the FastAPI endpoints over HTTP using TestClient backed by an in-memory SQLite database. Each test gets a fresh database through a pytest fixture, so tests are fully independent. In addition to automated tests, manual API testing is documented in API_TESTING_GUIDE.md using curl and Postman examples.

Test types

Unit tests

Unit tests cover the domain services directly without any HTTP layer or real database. They use GroupRepository and ExpenseRepository backed by plain Python dictionaries. This allows testing of all business rules in isolation.

Integration tests

Integration tests send real HTTP requests to the FastAPI application and verify response codes, payloads, and error messages. They use FastAPI's TestClient with a SQLite in-memory database, overriding the production PostgreSQL dependency via pytest fixtures.

Edge case tests

A dedicated test file covers scenarios that are easy to miss in normal flow: a user trying to leave a group with an unsettled debt, a group that has only one member, a user joining a group after expenses were already created, and numerical edge cases like amounts of $0.01.

Manual tests

The file API_TESTING_GUIDE.md documents a full set of curl commands that can be run against the live server to verify the complete user flow end to end, from registration to settling debts.

Test Files

Domain unit Tests:

- test_auth.py (11 tests) – user registration, login, JWT token validation, and the /auth/me endpoint.
- test_debt_calculation.py (9 tests) – debt matrix calculation, mutual debt netting, full and partial settlement, and settlement plan generation.
- test_expense_creation.py (9 tests) – expense creation with equal and custom splits, debtor validation, and penny rounding.
- test_expense_service.py (8 tests) – ExpenseService methods: splitting, netting, settle_up, drop_out_from_expense, and leave-with-debt guard.
- test_group_service.py (8 tests) – group creation, member invite, duplicate invite handling, and drop-out with and without unsettled debt.
- test_edge_cases.py (9 tests) – dine-and-dash prevention, single-member group, late joiner isolation, and small amount handling.
- test_settlement_simplification.py (10 tests) – circular debt cancellation, chain simplification, minimum transactions, and balance-zero verification.

API integration tests:

- test_auth_requests.py (6 tests) – HTTP-level registration, login, duplicate email, and /auth/me endpoint.
- test_expenses_requests.py (5 tests) – create expense with splits, paginated list, 403 for non-members, and 404 for unknown ID.
- test_groups_requests.py (6 tests) – create group, list groups, get ID and invite code, and auth guard.
- test_members_requests.py (6 tests) – list members, add by user ID, invite code endpoint, remove member, and error handling.
- test_debts_requests.py (4 tests) – list debts, settle a debt, double-settle returns 400, and unknown debt returns 404.
- test_health_requests.py – health check endpoint returns 200 with status ok.

Tools

The following tools are used across the test suite.

- pytest 8.3.5 – test runner and fixture management
- FastAPI TestClient – in-process HTTP testing without a live server
- SQLite (in-memory) – isolated database per test run
- Postman / curl – manual end-to-end testing documented in API_TESTING_GUIDE.md

Manual testing

The following scenarios describe how to manually verify that the main product flows work correctly

User registration and login

Verify that a new user can register, log in, and access their profile.

- Register a new user with a name, email, and password. Expected: the server returns a JWT token and a user object.
- Log in with the same credentials. Expected: a new token is returned successfully.
- Access the profile endpoint with the token. Expected: the correct user profile is returned.
- Access the profile endpoint without a token. Expected: access is denied.

Create a group and invite members

Verify that a user can create a group and invite other members. Start by registering three users - Alice, Bob, and Charlie.

- Alice creates a group with a name and currency. Expected: the group is created with a unique invite code.
- Alice invites Bob by his user ID. Expected: Bob is added as a group member.
- Alice invites Charlie by his email address. Expected: Charlie is added as a group member.
- View the group member list. Expected: all three users appear as members.

Create an expense and split

Verify that an expense is recorded correctly and the split amounts are calculated as expected.

- Alice creates a $150 dinner expense, paid by Alice, split equally between Bob and Charlie. Expected: the expense is created with two split records.
- View the expense list for the group. Expected: the new expense appears in the list.
- View the expense details. Expected: the full expense including both splits is returned.

View debts

Verify that the debt list correctly reflects what each member owes after an expense is created.

- View the group debts as Alice. Expected: two debt rows appear - Bob owes Alice $50 and Charlie owes Alice $50.
- View the group debts as a user who is not a member. Expected: access is denied.

Settle up

Verify that settling a debt marks it as paid and that settling the same debt twice is rejected.

- Settle Bob's debt. Expected: the server confirms the debt is settled.
- View the group debts again. Expected: Bob's debt no longer appears as outstanding.
- Try to settle Bob's debt again. Expected: the server returns an error indicating the debt is already settled.

Penny rounding

Verify that the application handles amounts that cannot be divided equally without floating-point errors.

- Alice creates a $10 expense split equally between Alice, Bob, and Charlie. Mathematically each share is $3.333. Expected: the app handles the rounding correctly and the total of all shares equals $10 without any amount disappearing.
- View the group debts. Expected: Bob owes Alice approximately $3.33 and Charlie owes Alice approximately $3.33.

Circular debt simplification

Verify that the settlement algorithm correctly identifies when circular debts cancel each other out.

- Create three expenses of equal amount: Alice pays $10 and only Bob is the debtor, Bob pays $10 and only Charlie is the debtor, Charlie pays $10 and only Alice is the debtor. Expected: all three debts cancel each other out completely.
- View the group debts. Expected: the debt list is empty - no one owes anyone anything.

Dine and dash prevention

Verify that a user with an unsettled debt cannot simply be removed from the group, leaving the balance unresolved.

- Bob owes Alice $50 and has not settled. Try to remove Bob from the group. Expected: the application prevents the removal and returns an error indicating Bob has unsettled debts.
- Settle Bob's debt first, then remove him from the group. Expected: Bob is removed successfully and the member list updates.